Connect Four

Packages
- Python
- Rich
- Numpy
- PyQt

- PyQt for GUI
- Rich for colour terminal text

Requirements

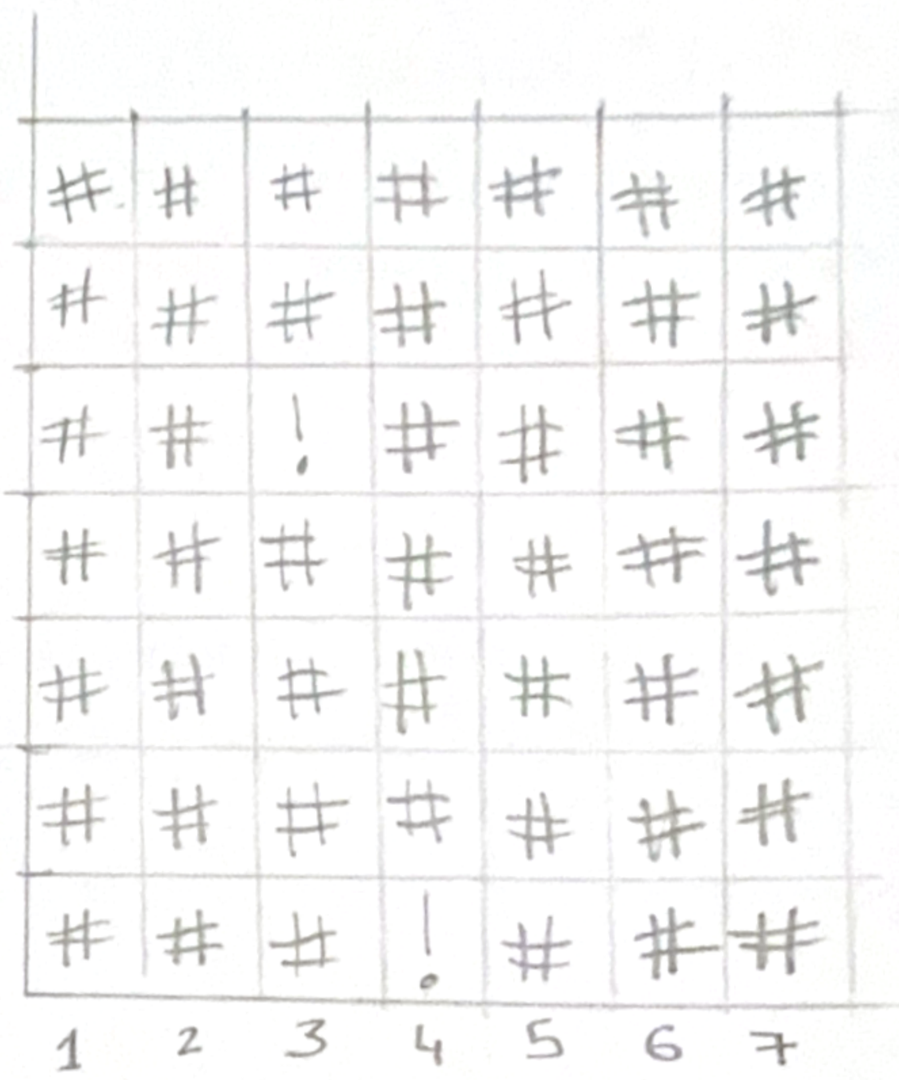
* Basic version is terminal based.
* Customisable board size - min of 6x7
* Blocked Cells - min 1 cell, max 10% total cells.
* Player vs Player + Player vs Computer

Game Logic

* Only check for win after 7 turns.
* Win check all 4 directions
* Compare total possible moves with count of moves made to check for draw

TERMINAL BASED

* use # to show empty cells.
* Use ! to show blocked cells.
* Use X and O as game pieces.
* When asking for column input, use 1-based indexing and convert to 0-based.
* Only ask for col input as Pieces have gravity.
* Since pieces fall to bottom, scan from bottom.
*



AI Agent

* 3? difficulty levels
  - randomly placed
  - monte carlo tree search.
  - machine learning model integration

Rich Colours normalisation

* Borders + Title = Pink.
* Info + Prompts = Gold
* text = Cyan
* Board blank cell = blue
* Board blocked cell = red
* Board X game piece = purple
* Board O game piece = yellow
* Board show win line = green
* win text = green

Monte Carlo Tree Search

Within the context of dynamic Connect 4 board size.

4 Stages of the algorithm:

- Stage 1: Selection (Exploration)
- Stage 2: Expansion
- Stage 3: Simulation
- Stage 4: Backpropegation

UCB1 formula:

$$UCB1(i) = \bar{X}_i + c\sqrt{\frac{\ln N}{n_i}}$$

- $\bar{X}$ is average reward of node i
- C is exploration parameter ($\sqrt{2}$)
- N is total number of visits to parent node
- n_i is number of visits to node i

Stage 1 - Assume a board size of 6x7

The AI checks all 7 columns. It begins with column 1. This col has 0 visits so far. - no game pieces in col - For this reason, it will have a UCB1 score of infinity. MCTS will always try every untried column at least once before it begins doubling down.

Stage 2

The AI then imagines dropping a game piece into colomn 1. It then adds this board game state to its mental tree so it can test what happens next

Stage 3:

The AI then simulates a random game from this hypothetical board state. It plays the game by randomly dropping game pieces. It does this until either game piece wins or a draw occours.

Stage 4

After the simulation ends, the AI walks backwards from the leaf to the root, incrementing visit counts (+1) and updating win scores (+1 for the player who made the move or +0.5 for a draw) for every node along the ancestory path.

Final Decision - Choosing the most robust child

After all iterations (2500 practice games) are complete, the AI ignores the UCB1 formula, looks directly at the children attached to the root node, and picks the col with the highest visit count - This is because it has survived the most simulated stress tests.

Implementing Monte Carlo Tree Search

Before writing the MCTS code, a quick way for the AI to query which cols can accept a piece is needed. The standard 6x7 board has 4.5×10^{12} board states, and each turn branches into at most 7 legal column drops.

To implement, drop the row and col and just use col, as pieces are bound by gravity. To get valid locations:

```
def get_valid_locations(board):
    valid_locations = []
    cols = board.shape[1]
    for col in range(cols):
        if "#" in board[:,col]:
            valid_locations.append(col)
    return valid_locations
```

Since random cells are blocked with "!", if a blocked cell sits at row 0, checking only board[0, col] for "#" would falsely mark the whole col as full even if empty "#" slots exist underneath. So we scan the full numpy column slice board[:,col] to handle obstacles safely.

```
# ! 0 # #   col 3 is invalid, while col 2 is valid
# # x # !
# # 0 # #
# 0 x x 0
1 2 3 4 5
```

MCTSNODE and Lazy Expansion

Each node stores its local scorecard (self.visits, self.wins), its parent link (self.parent), the column move that created it (self.column) and a queue of unvisited columns (self.untried_moves)

Had to use this to avoid right-to-left bias. Without it, get_valid_locations returns [0,1,2,3,4,5,6...] Since .pop() removes the last element, the AI will always expand the last col first and so on. So by shuffling it eliminates potential bias

```
class MCTSNode:
    def __init__(self, board, parent, col, game_piece):
        self.board = board
        self.parent = parent
        self.column = col
        self.game_piece = game_piece
        self.visits = 0
        self.wins = 0.0
        self.untried_moves = get_valid_locations(board)
        random.shuffle(self.untried_moves)
```

```
def expand(self, next_game_piece):
    col = self.untried_moves.pop()
    new_board = self.board.copy()
    drop_piece(new_board, next_game_piece, col)
    child_node = MCTSNode(board = newboard,
                          column = col, game_piece = next_game_piece)
    self.children.append(child_node)
    return child_node
```

Using NumPy's .copy() instead of Pythons .deepcopy() cuts memory cloning overhead by ~10x across 1500 expansions.

Best child via UCB1

```
def is_fully_expanded(self):
    return len(self.untried_moves) == 0
```

once this function returns True, the MCTS uses the upper confidence Bound 1 (UCB1) to balance exploitation (high win rate cols) and exploration (optimism in the face of uncertainty for rarely visited columns)

$$UCB1(child) = \frac{child.wins}{child.visits} + c\sqrt{\frac{\ln(self.visits)}{child.visits}}$$

C is usually $\sqrt{2}$, which is 1.414

```
def best_child(self, c=1.414):
    for child in self.children:
        if child.visits == 0:
            return child

    def ucb(child):
        exploit = child.wins / child.visits
        explore = c * math.sqrt(math.log(self.visits)/child.visits)
        return exploit + explore

    return max(self.children, key=ucb)
```

Prevents division by 0 error; guarantees every child gets ≥1 rollout.

Empirical win rate (Exploiation)

Theoretical exploration constant from Hoeffding's inequality

Wiring AIAgent.mcts_search()

```
def mcts_search(self, board):
    root = MCTSNode(board=board.copy(), parent=None,
                    column=None, game_piece=self.human_piece)
    for _ in range(self.iterations):
        node = root
        current_piece_turn = self.game_piece
        while node.is_fully_expanded and not node.is_terminal(self.game_piece):
            node = best_child(c=1.414)
            current_piece_turn = self.human_piece if node.game_piece != self.human_piece else self.game_piece
        if not node.is_fully_expanded and not node.is_terminal(self.game_piece):
            node = node.expand(current_piece_turn)
        winner = self.random_rollout(node)
        self.backpropogate(node, winner)
    if not root.children:
        valid = get_valid_locations(board)
        return random.choice(valid)
    robust_child = max(root.children, key=lambda child: child.visits)
    return robust_child.column
```

Phase 1

Phase 2

Phase 3

Phase 4

Not in for loop

Once all iterations finish, discard UCB1 score and Pick child with most visits.